

SENTINEL

AI-Powered Financial Crime Investigation & Threat Intelligence Platform

COMPLETE HACKATHON PROJECT DOCUMENTATION

Tagline: "We don't just detect fraud. We investigate it."

A complete blueprint covering requirements analysis, functional and non-functional requirements, use cases, DFD, ER diagram, architecture, database design, AI agents, APIs, security, testing, deployment, outcomes, future scope, team roles, judging strategy and the final demo plan.

Primary Track: CyberSec **Secondary:** AI/ML + FinTech + Web3/Open Innovation

1. Executive Summary

SENTINEL is an AI-assisted investigation platform for cyber-financial crime. It accepts fragmented evidence such as screenshots, URLs, PDFs, messages, QR codes and transaction records, extracts useful entities, performs threat and financial analysis, constructs an investigation graph, produces an explainable risk assessment and generates an investigation report.

Core idea: Traditional fraud detection asks “Is this suspicious?”. Sentinel asks “What happened, how are the entities connected, what evidence supports the conclusion, where did the money move, and what should the investigator do next?”

2. Problem Statement

- Fraud evidence is scattered across messages, screenshots, URLs, transaction records and account information.
- Traditional systems frequently focus on detection rather than complete investigation.
- Investigators spend significant time manually correlating entities and transaction chains.
- Risk scores without explanations are difficult to trust.
- Evidence needs an integrity mechanism so investigators can detect later modification.
- Manual report creation consumes time that could be spent on investigation.

3. Proposed Solution

Sentinel creates a single investigation workspace in which multimodal AI, threat intelligence, financial analytics, graph reasoning and evidence integrity work together.

Evidence → AI extraction → threat analysis → financial analysis → entity resolution → graph construction → investigation reasoning → explainable risk score → recommended actions → report → evidence hash verification.

4. Requirements Analysis

4.1 Functional Requirements

ID	Requirement
FR-01	User registration, login, logout and role-based access.
FR-02	Create, update, assign, search and close investigation cases.
FR-03	Upload screenshots, PDFs, text, URLs, QR codes and transaction evidence.
FR-04	Extract entities such as phone, email, URL, domain, UPI, account, amount and wallet.
FR-05	Analyze URLs/domains/messages for phishing and threat indicators.
FR-06	Analyze transactions for amount, velocity, beneficiary and chain anomalies.
FR-07	Construct an interactive entity relationship investigation graph.
FR-08	Generate an explainable risk score with supporting factors.
FR-09	Generate investigation summaries and downloadable reports.
FR-10	Hash evidence and store the hash on a test/local blockchain.
FR-11	Verify whether the current evidence matches its registered hash.
FR-12	Maintain case history, findings, actions and audit information.

4.2 Non-Functional Requirements

Category	Requirement / Target
Performance	Normal API requests should feel responsive; AI jobs may run asynchronously.
Scalability	Architecture should support horizontal scaling of API and AI workers.
Security	JWT/refresh-token authentication, RBAC, validation, rate limiting and secure file handling.
Reliability	External AI/API failure should not destroy a case; use cached/synthetic fallback data for the demo.
Usability	Investigator should understand a case, evidence and risk factors quickly.
Maintainability	TypeScript, modular services, documented APIs and environment-based configuration.
Observability	Structured logs, health endpoints and basic request/error monitoring.
Privacy	Use synthetic or explicitly permitted demo data during the hackathon.

4.3 Hardware / Software Requirements

Item	Minimum practical requirement
Developer machine	8 GB RAM; 16 GB recommended for Docker + databases + browser.
OS	Windows/Linux/macOS; Docker Desktop on Windows is acceptable.
Runtime	Node.js 20+; Python 3.11+ only if a separate AI service is used.
Databases	PostgreSQL and Neo4j; Redis optional but recommended.
Tools	Git, Docker, VS Code, Postman/Insomnia, browser.
AI	Multimodal LLM API with available quota, or a local/mock fallback for the demo.
Blockchain	Hardhat local network; no real crypto required.

5. Stakeholder & Actor Analysis

Actor	Purpose
Investigator	Creates cases, uploads evidence, reviews findings and takes actions.
Analyst	Reviews financial and threat intelligence signals.
Administrator	Manages users, roles, system settings and audit information.
Victim / Reporter	Provides suspicious evidence to initiate an investigation.
External Intelligence Provider	Provides optional threat/domain intelligence.
AI Model	Extracts, classifies, summarizes and reasons over evidence.

6. Use Case Model

Primary use cases are Login, Manage Case, Upload Evidence, Analyze Evidence, Investigate Case, View Graph, Review Risk, Trace Transactions, Generate Report and Verify Evidence.

Use Case	Actor	Result
Login	User	Authenticated session and role.
Create Case	Investigator	New investigation workspace.
Upload Evidence	Investigator	Evidence stored and queued for analysis.
Analyze Evidence	Investigator / AI	Entities, indicators and extracted data.
Investigate Case	Investigator / AI	Correlated findings and attack narrative.
View Graph	Investigator	Connected entities and relationships.
Generate Report	Investigator	Investigation report with findings and recommendations.

Use Case	Actor	Result
Verify Evidence	Investigator / Auditor	Hash verification result.

7. Data Flow Diagrams

7.1 Level 0 — Context Diagram

Investigator / Analyst → SENTINEL → investigation results, reports and verification status. The platform communicates with databases, AI services and a blockchain network.

[illegible]

7.2 Level 1 — Processing Flow

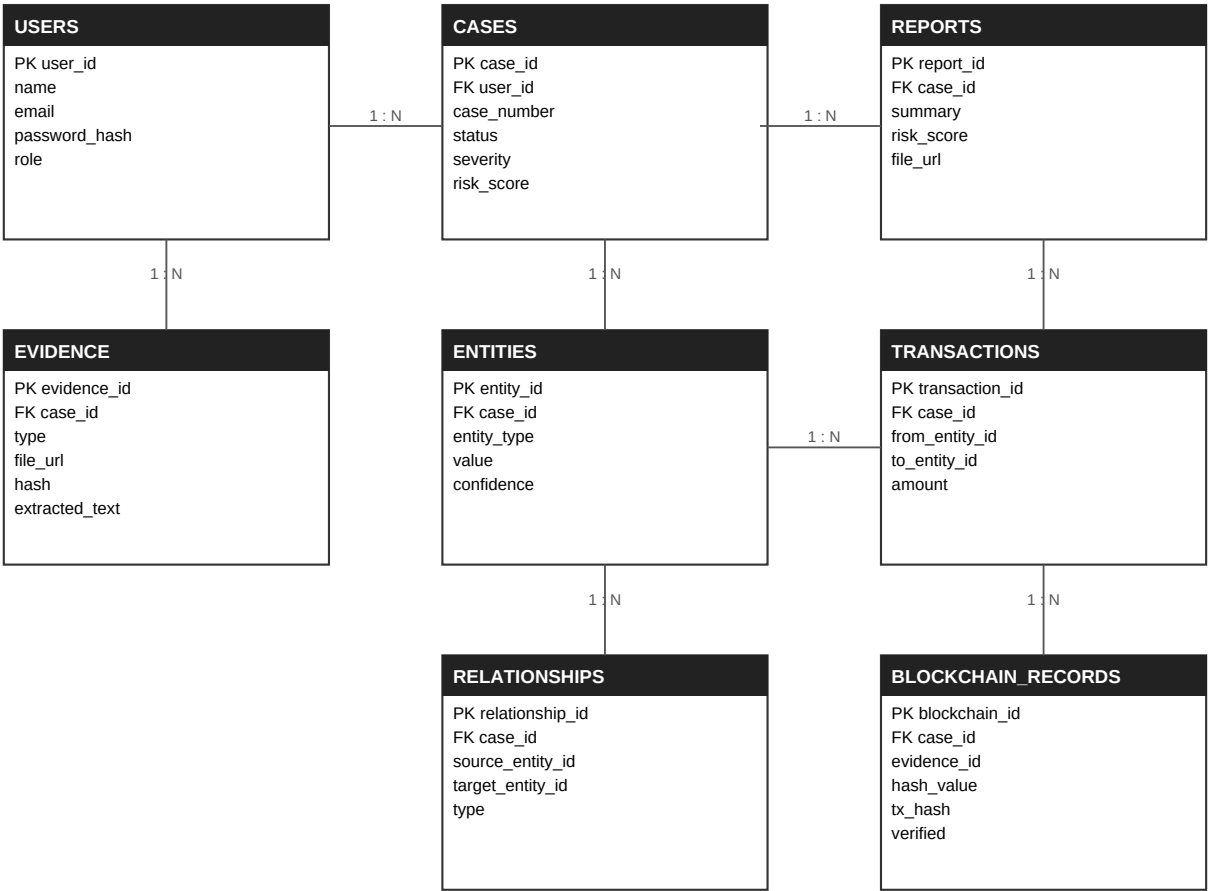
User → Authentication → Case Management → Evidence Upload → Evidence Processing → AI Extraction → Threat Analysis + Financial Analysis → Graph Construction → Investigation Agent → Risk Scoring → Report Generation → Evidence Hash / Verification.

7.3 Level 2 — Evidence Processing

Upload → File validation → OCR/text extraction → multimodal LLM extraction → schema validation → entity normalization → persistence → graph update → findings.

8. ER Diagram

The relational ER model stores structured application data in PostgreSQL. Relationship-heavy investigation data is also projected into Neo4j for fast graph traversal and visualization.



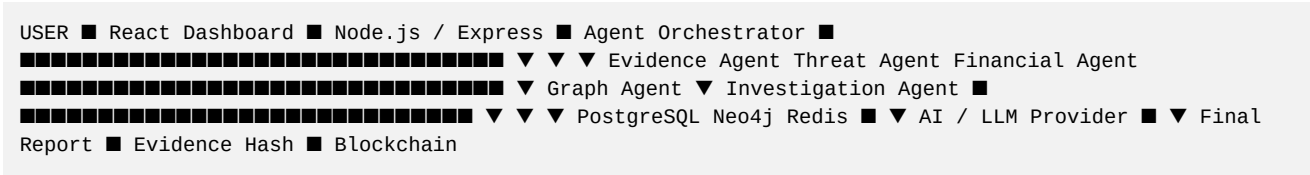
Key relationships: USERS 1:N CASES; CASES 1:N EVIDENCE; CASES 1:N ENTITIES; CASES 1:N TRANSACTIONS; CASES 1:N REPORTS; CASES 1:N BLOCKCHAIN_RECORDS; ENTITIES are connected through RELATIONSHIPS.

9. Database Design

Table	Important columns	Purpose
users	user_id, name, email, password_hash, role	Authentication and authorization.
cases	case_id, user_id, case_number, status, severity, risk_score	Investigation case lifecycle.
evidence	evidence_id, case_id, type, file_url, hash, extracted_text	Raw evidence and extracted content.
entities	entity_id, case_id, entity_type, value, confidence	Normalized investigation entities.
relationships	relationship_id, source_entity_id, target_entity_id, type	Entity connections.
transactions	transaction_id, case_id, from_entity_id, to_entity_id, amount	Financial transactions.
reports	report_id, case_id, summary, risk_score, file_url	Generated investigation reports.
blockchain_records	blockchain_id, evidence_id, hash_value, tx_hash, verified	Evidence integrity records.

10. System Architecture

Recommended hackathon architecture: React/Vite frontend, Node.js/Express API, LangGraph orchestration, PostgreSQL/Prisma for structured data, Neo4j for the investigation graph, Redis for caching/job state, a multimodal LLM for AI tasks, and Hardhat for local blockchain evidence verification.



10.1 Why Two Databases?

PostgreSQL: users, cases, evidence metadata, transactions and reports. **Neo4j:** relationships such as victim→phone→domain→UPI→account→wallet. This separation is justified by the different access patterns rather than being technology for technology's sake.

11. AI Agent Design

Agent	Input	Output
Evidence Agent	Images, PDFs, messages, URLs	Entities, text, claims, indicators.
Threat Agent	URLs, domains, IP/email indicators	Threat factors, confidence, threat score.
Financial Agent	Transactions, amounts, timestamps	Anomaly factors, transaction risk.
Graph Agent	Entities + relationships	Investigation graph and connected paths.
Investigation Agent	All agent findings	Narrative, final risk, recommendations and report.

11.1 Structured AI Output

```
{ "entities": [], "indicators": [], "transactions": [], "claims": [], "confidence": 0.94 }
```

Use strict schemas and validation. The LLM should produce structured findings rather than uncontrolled prose whenever possible.

12. Explainable Risk Engine

A transparent weighted model can combine independent signals. Example weighting:

Signal	Weight
Threat intelligence	25%
Transaction anomaly	25%
Entity risk	20%
Graph risk	15%
Message/evidence indicators	15%

Risk = 0.25T + 0.25F + 0.20E + 0.15G + 0.15M. The exact formula should be calibrated using your synthetic validation dataset.

Example UI: 92/100 — HIGH. Reasons: suspicious domain, impersonation language, unusual amount, new beneficiary, rapid fund movement and multiple connected suspicious entities.

13. Blockchain Evidence Integrity

```
Do not store private evidence on-chain. Calculate a cryptographic hash of the evidence and register the hash in a local/test blockchain smart contract. Later, recompute the hash and compare it with the
```

registered value.

Evidence → SHA-256 → hash → smart contract → transaction hash → verification.

Important: blockchain verification demonstrates integrity of the registered bytes/hash; it does not prove that the original evidence is truthful or that a person is guilty.

14. API Specification

Module	Endpoints
Auth	POST /api/auth/register; POST /api/auth/login; POST /api/auth/refresh; POST /api/auth/logout
Cases	GET /api/cases; POST /api/cases; GET /api/cases/:id; PATCH /api/cases/:id
Evidence	POST /api/cases/:id/evidence; GET /api/cases/:id/evidence; GET /api/evidence/:id
AI	POST /api/cases/:id/analyze; GET /api/cases/:id/findings; POST /api/cases/:id/investigate
Graph	GET /api/cases/:id/graph; GET /api/entities/:id/connections
Reports	POST /api/cases/:id/report; GET /api/cases/:id/reports
Blockchain	POST /api/evidence/:id/hash; POST /api/evidence/:id/verify

15. Frontend / UX Design

Page	Main content
/login	Authentication.
/dashboard	Case KPIs, critical cases, alerts, risk distribution.
/cases	Case list, filters, search and severity.
/cases/new	Create case and initial evidence.
/cases/:id	Case overview, risk score, timeline and findings.
/cases/:id/evidence	Evidence gallery and extracted entities.
/cases/:id/investigation	AI investigation findings and narrative.
/cases/:id/graph	Interactive relationship graph.
/cases/:id/transactions	Transaction chain and anomaly indicators.
/cases/:id/report	Generated report and export.

15.1 The Main “Wow” Screen

Case ID + severity + 0–100 risk score + investigation graph + evidence panel + AI findings + transaction trail + recommended actions should be visible in one coherent workspace.

16. Security Requirements

- Use JWT access tokens with refresh tokens and secure expiration policies.
- Use bcrypt/Argon2 for password hashing; never store plaintext passwords.
- Enforce role-based access control on case and evidence APIs.
- Validate MIME type, extension and file size before processing uploads.
- Sanitize extracted text before rendering it in the UI.
- Use parameterized queries/ORM APIs and validate all input.
- Add API rate limiting and basic abuse protection.
- Do not expose secrets in source control; use environment variables.
- Keep an audit trail for sensitive case actions.
- Use only synthetic or explicitly authorized data for the hackathon.

17. Testing Strategy

Test type	Examples
Unit	Risk formula, entity normalization, validators, authentication helpers.
Integration	API ↔ PostgreSQL, API ↔ Neo4j, AI workflow ↔ persistence.
AI evaluation	Normal SMS, phishing SMS, fake URL, normal transaction, suspicious transaction chain.
Security	Unauthorized access, invalid JWT, malicious upload, XSS, injection, rate limiting.
UI	Upload flow, graph rendering, report generation, error states.
Demo reliability	LLM timeout, API outage, empty evidence, malformed data; verify fallback path.

18. Synthetic Dataset Design

For a hackathon, use a controlled synthetic dataset so the demo is repeatable and safe.

Object	MVP target
Cases	20
Evidence items	50
Entities	100
Relationships	200
Transactions	300

Build one polished end-to-end scenario and several normal/negative cases for comparison. Mark all demo records as **SIMULATED DATA — HACKATHON DEMONSTRATION**.

19. Showstopper Demo Script

Time	Action	Judge sees
0:00–0:30	Problem statement	Why detection alone is insufficient.
0:30–1:00	Upload scam screenshot	Evidence enters the case.
1:00–1:30	AI extraction	URL, phone, UPI, amount and claims extracted.
1:30–2:00	Threat + financial analysis	Risk factors appear.
2:00–2:30	Open graph	Victim → phone → domain → UPI → account → wallet.
2:30–3:00	Generate report	Evidence-backed narrative and recommendations.
3:00–3:20	Verify hash	Blockchain integrity result.

Final pitch line

“Existing fraud systems ask whether something is suspicious. Sentinel reconstructs what happened, connects the entities, shows where the money moved, explains the evidence and tells the investigator what to do next.”

20. Technology & Cost Analysis

Technology	Purpose	Hackathon cost
React + Vite + Tailwind	Frontend	Free/open source
Node.js + Express + TypeScript	Backend	Free/open source
PostgreSQL + Prisma	Structured data	Free locally
Neo4j Community	Graph investigation	Free locally
Redis	Cache/job state	Free locally
LangChain + LangGraph	AI orchestration	Open source
Tesseract OCR	OCR	Free/open source
Hardhat + Solidity	Local blockchain	Free
Docker + Compose	Packaging	Free for hackathon use
LLM API	Multimodal reasoning	Depends on provider/quota
Cloud hosting	Optional deployment	Can be avoided or use suitable free tier

Budget target: ■0–■2,000 for the hackathon is realistic if you run databases and blockchain locally and use an available/free LLM quota. The largest variable cost is AI API usage.

21. 36-Hour Development Plan

Hours	Deliverable
0–4	Architecture, schema, UI wireframe, synthetic demo scenario.
4–10	React dashboard, authentication, case management and evidence upload.
10–18	AI extraction, threat/financial analysis and structured outputs.
18–25	Neo4j graph, graph visualization, investigation workflow.
25–30	Report generation, evidence hash and blockchain verification.
30–34	UI polish, animations, errors, loading states, test data.
34–36	Full rehearsal, offline/fallback path, PPT and judge Q&A.

22. Team Responsibilities

Member	Primary responsibility
Developer 1	Backend, API, PostgreSQL, Prisma and authentication.
Developer 2	AI agents, LangGraph, prompts, extraction and risk engine.
Developer 3	React UI, dashboard, case workspace and graph visualization.
Developer 4	Neo4j, blockchain, Docker, testing, security and DevOps.

23. Expected Outcomes

- A unified investigation workspace instead of disconnected fraud alerts.
- Automatic extraction and normalization of useful evidence.
- Relationship graph showing how entities and transactions are connected.
- Explainable risk assessment based on multiple signals.
- Automated investigation narrative and recommended actions.
- Tamper-evident evidence registration using cryptographic hashes.
- Significant reduction in repetitive manual investigation and report preparation work.
- A strong foundation that can later evolve into a production-scale fraud/SOC platform.

23.1 Success Metrics

Metric	Prototype target
Entity extraction accuracy	>90% on the synthetic evaluation set
Demo case completion	>95% successful runs
Report generation	>95% successful runs
Graph construction	>95% successful runs
Investigation time reduction	>70% compared with the team's manual baseline

24. Limitations & Responsible Claims

- Synthetic data does not represent all real-world fraud behavior.
- LLM outputs can be wrong; human investigator review remains necessary.
- Threat-intelligence APIs may have availability and rate limits.
- A risk score is an investigative aid, not proof of criminal activity.
- Blockchain proves integrity of a registered hash, not the truthfulness of the evidence itself.
- Production banking integrations require regulatory, privacy and security controls beyond a hackathon MVP.

25. Future Scope

- Real-time transaction streaming using Kafka.
- Advanced graph analytics and community detection.
- Integration with authorized threat-intelligence feeds.

- Real-time alerting and investigator collaboration.
- Multilingual and voice-based evidence processing.
- SIEM/SOC integration.
- Bank/FinTech integration through secure APIs.
- Kubernetes-based production deployment with observability.
- Human-in-the-loop approval workflows.
- Continuous model evaluation and feedback loops.

26. What NOT to Build During the Hackathon

- A generic chatbot over PDFs.
- A simple fraud/not-fraud classifier with no investigation workflow.
- A basic payment app with blockchain added only for decoration.
- A full microservice/Kubernetes platform before the core demo works.
- A huge feature list with no reliable end-to-end scenario.

Priority: Build one complete vertical slice first — upload evidence → AI extraction → risk → graph → report. Only then add blockchain and optional features.

27. Judge-Focused Differentiators

Judging dimension	Sentinel answer
Innovation	Investigation, not detection alone; combines multimodal evidence, graph reasoning and financial analysis.
Technical depth	Agent orchestration + LLM + PostgreSQL + Neo4j + blockchain integrity.
Demo impact	A single screenshot can trigger a complete investigation workflow.
Explainability	Risk score is accompanied by evidence-backed factors.
Business value	Applicable to banks, FinTechs, enterprises and investigation teams.
Reliability	Synthetic controlled dataset and fallback paths make the demo repeatable.

28. Final Product Definition

Project: SENTINEL — AI-Powered Financial Crime Investigation & Threat Intelligence Platform

Core innovation: An agentic AI investigation engine that transforms fragmented cyber-financial evidence into an explainable investigation graph, risk assessment, transaction trail and evidence-backed report.

ONE SCREENSHOT → AI UNDERSTANDS → ENTITIES EXTRACTED → THREATS ANALYZED → TRANSACTIONS CONNECTED → GRAPH GENERATED → ATTACK RECONSTRUCTED → RISK EXPLAINED → REPORT GENERATED → EVIDENCE VERIFIED

“One screenshot. Full investigation.”

Appendix A — Recommended Repository Structure

```
sentinel/ ■■■ frontend/ ■ ■■■ src/components/ ■ ■■■ src/pages/ ■ ■■■ src/services/ ■ ■■■ src/hooks/
■■■ backend/ ■ ■■■ src/controllers/ ■ ■■■ src/routes/ ■ ■■■ src/services/ ■ ■■■ src/agents/ ■ ■■■
src/middleware/ ■ ■■■ prisma/ ■■■ ai-service/ # optional separate service ■■■ blockchain/ ■ ■■■
contracts/ ■ ■■■ scripts/ ■ ■■■ test/ ■■■ data/synthetic/ ■■■ docs/ ■■■ docker-compose.yml ■■■
.env.example ■■■ README.md
```

Appendix B — Recommended Build Order

1. Database schema + backend health check.
2. Case creation + evidence upload.
3. AI extraction with structured JSON.
4. Risk engine.
5. Neo4j graph creation and visualization.
6. Investigation agent and report.
7. Blockchain evidence hash.
8. Security hardening and tests.
9. UI polish and final demo rehearsal.